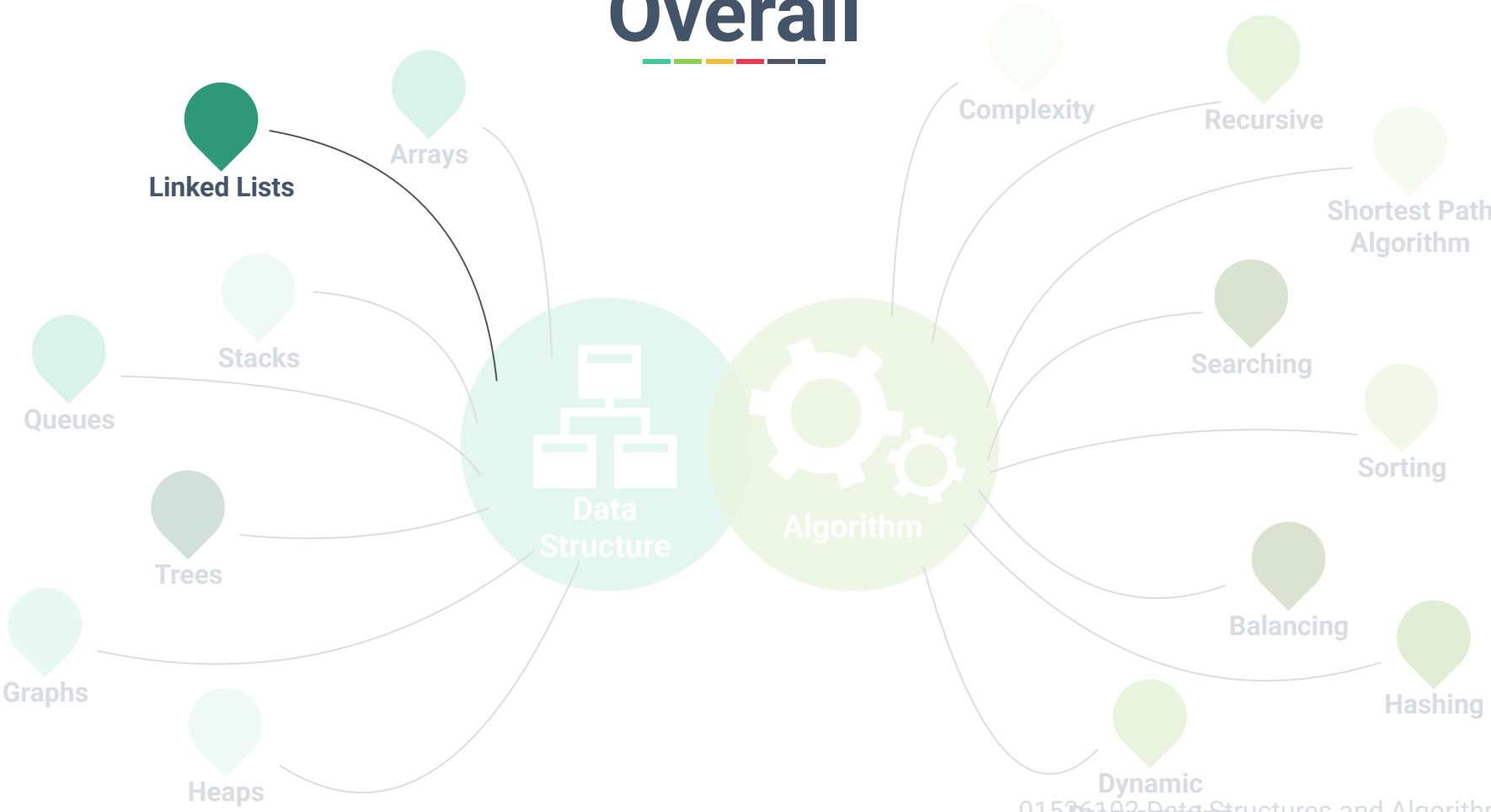
Chapter 5: Linked Lists

Dr. Sirasit Lochanachit

Overall


Linked Lists
Arrays
Stacks
Queues
Trees
Graphs
Heaps
Data Structure
Algorithm
Complexity
Recursive
Shortest Path Algorithm
Searching
Sorting
Balancing
Hashing
Dynamic Programming

Today's Outline

1. What is a Linked List?
2. Singly Linked Lists
    - Traversing
    - Insert a node
    - Delete a node

Previously

Python's array-based list

- Stack
- Queue

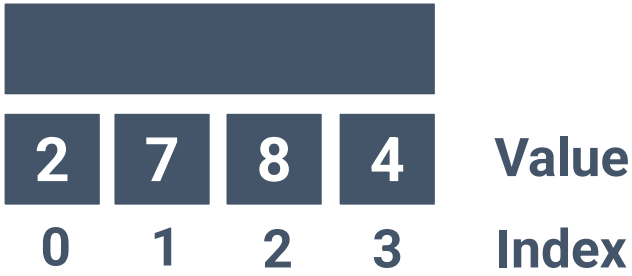
Disadvantages of array:

- Length of array has to be pre-allocated, empty space wasted.
- Adding or removing elements between values in the array is expensive - $O(n)$

Linked Lists

To avoid these limitations, an alternative to array is **linked list**.

Array



Linked lists

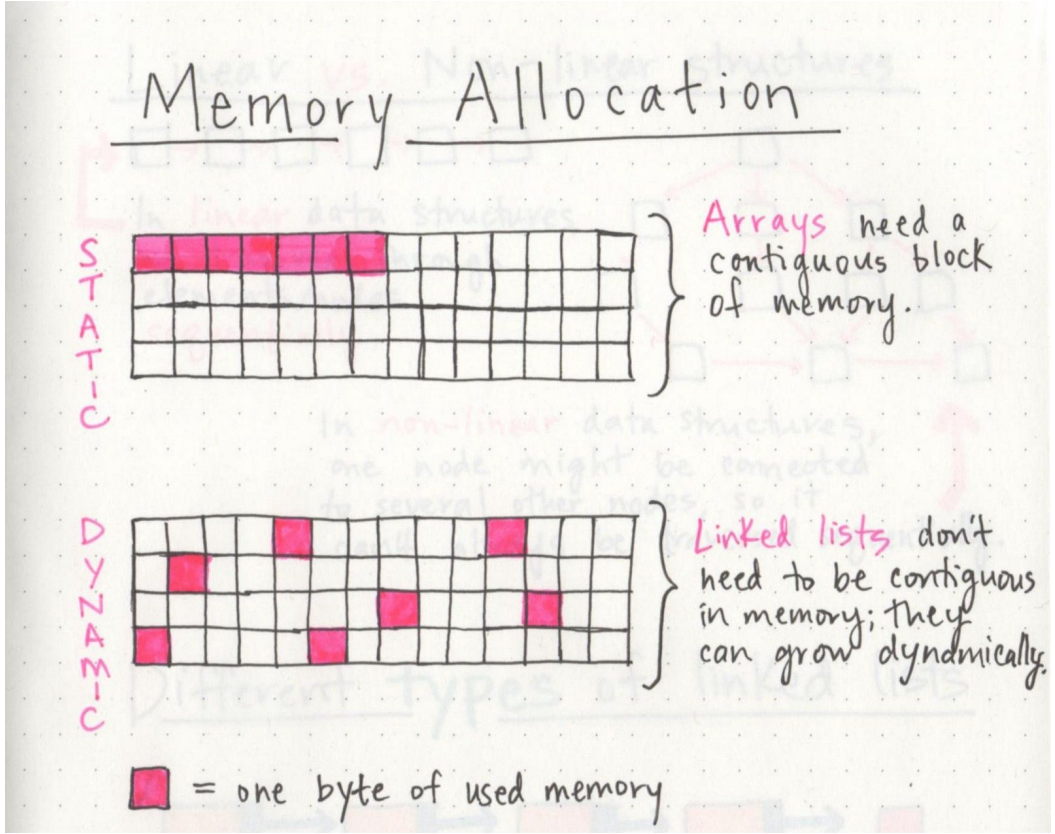
Linked Lists

Static

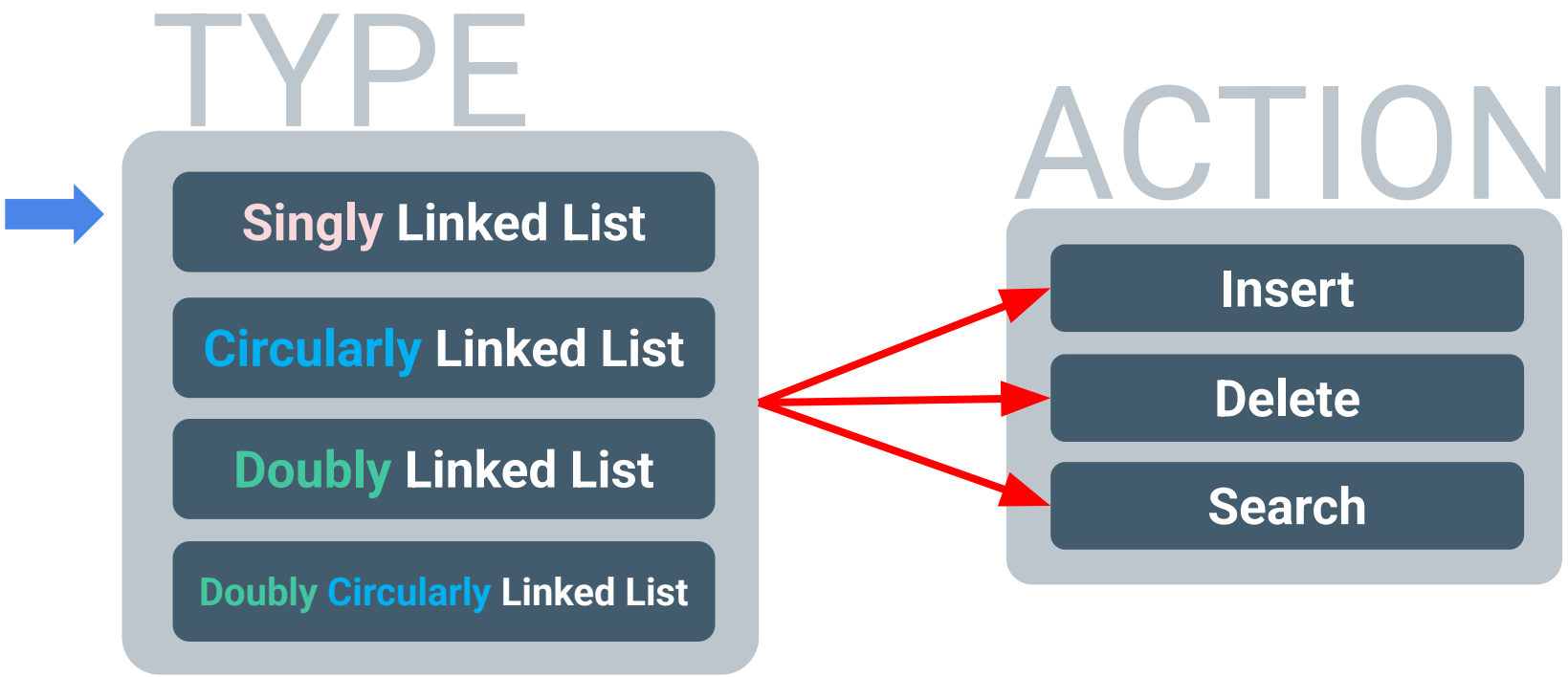
- Pre-allocated
- Fixed size
  - Unable to grow

Dynamic

- Allocated as needed
- Able to grow



Linked Lists



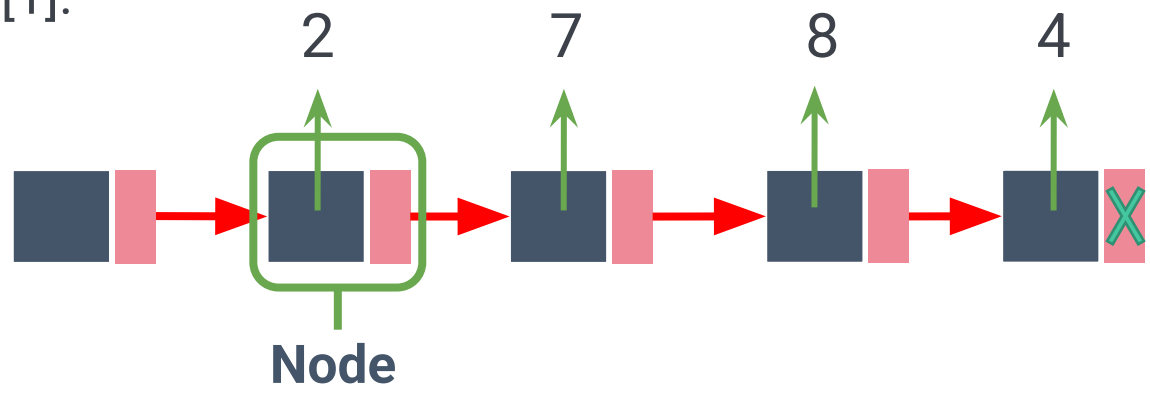
What is a Linked List?

A singly **linked list** is a collection of nodes that form a linear order of a sequence [1].

[1] Michael T. Goodrich et al., Data Structures and Algorithms in Python, 2013

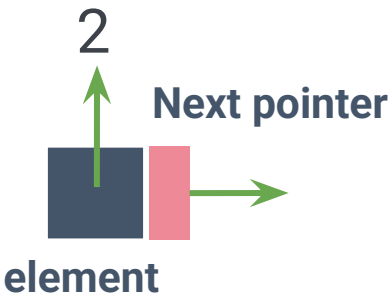
What is a Linked List?

A singly **linked list** is a collection of nodes that form a linear order of a sequence [1].



[1] Michael T. Goodrich et al., Data Structures and Algorithms in Python, 2013

Linked List Node



Linked List Node Structures

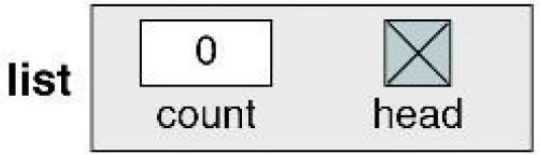
Create a Linked List

1. **Create a sentinel/header/root node**

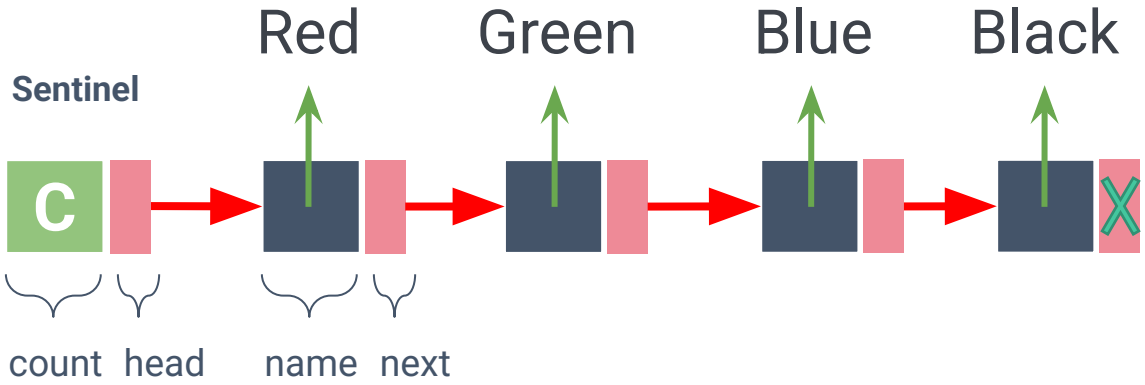
Algorithm createList (list)

1. Allocate a list
2. Set list head to null
3. Set list count to 0

End createList



Singly Linked Lists



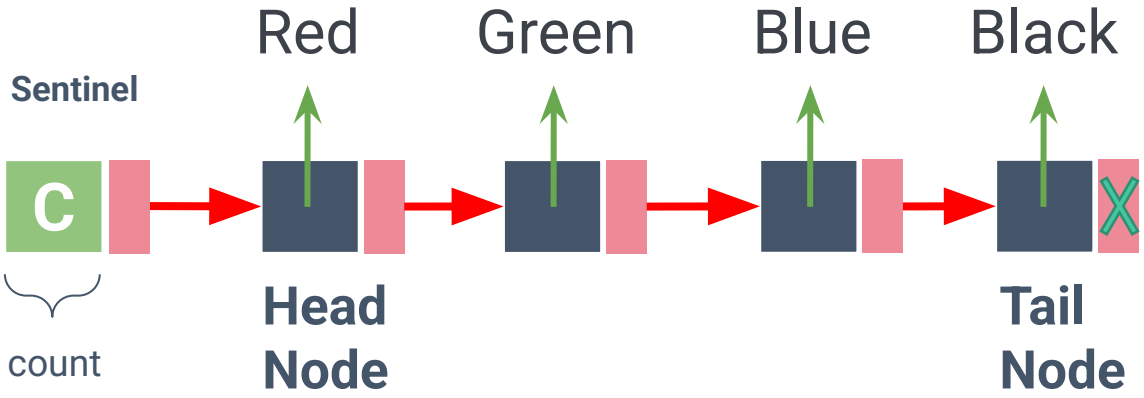
Color

String name

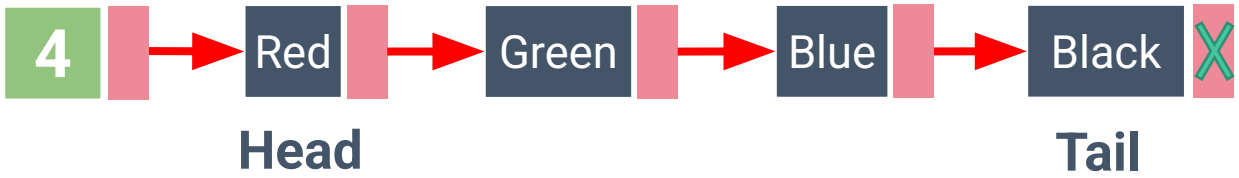
Color next

End Color

Singly Linked Lists



Singly Linked Lists



Create a Linked List

2. Create a data/element Node

```
Algorithm createDataNode (d, p)
    colorNew = allocate(Color)
    name = d
    next = p
    return colorNew
End createDataNode
```

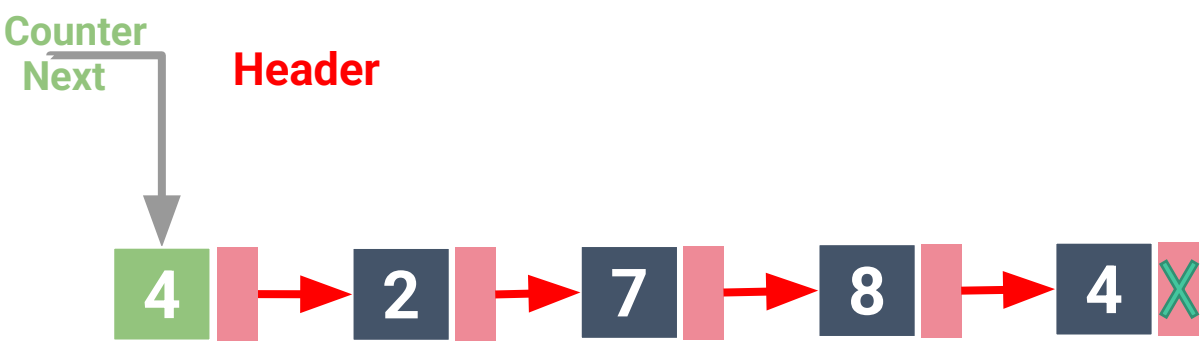
Traversing Singly Linked Lists

Address/ Byte#	Value
6000	4
6001	6002
6002	2
6003	6008
6004	8
6005	6012
6006	
6007	
6008	7
6009	6004
6010	
6011	
6012	4
6013	None

Suppose that it takes 1 byte to store an integer.

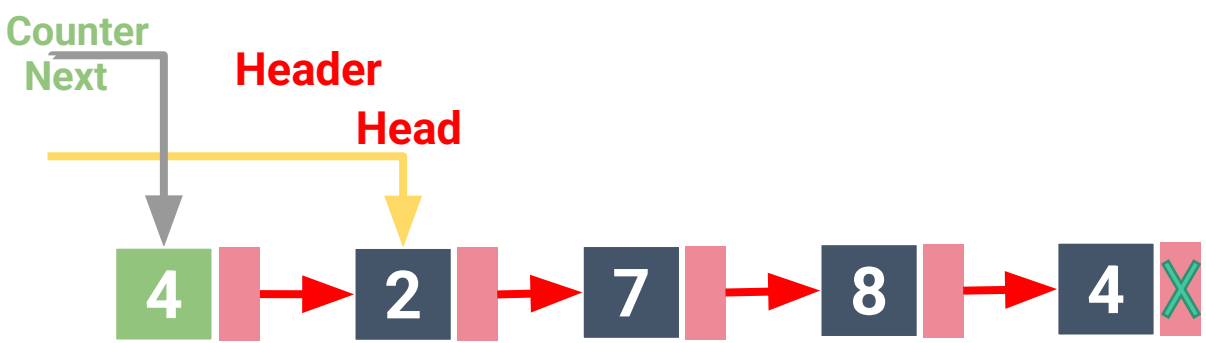
Traversing Singly Linked Lists

Address/ Byte#	Value
6000	4
6001	6002
6002	2
6003	6008
6004	8
6005	6012
6006	
6007	
6008	7
6009	6004
6010	
6011	
6012	4
6013	None



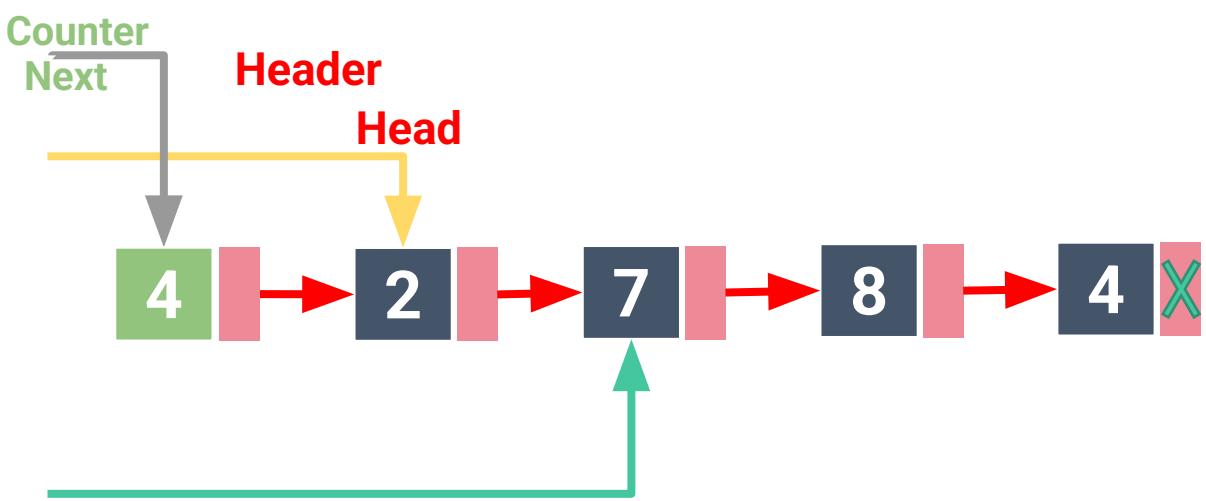
Traversing Singly Linked Lists

Address/ Byte#	Value
6000	4
6001	6002
6002	2
6003	6008
6004	8
6005	6012
6006	
6007	
6008	7
6009	6004
6010	
6011	
6012	4
6013	None



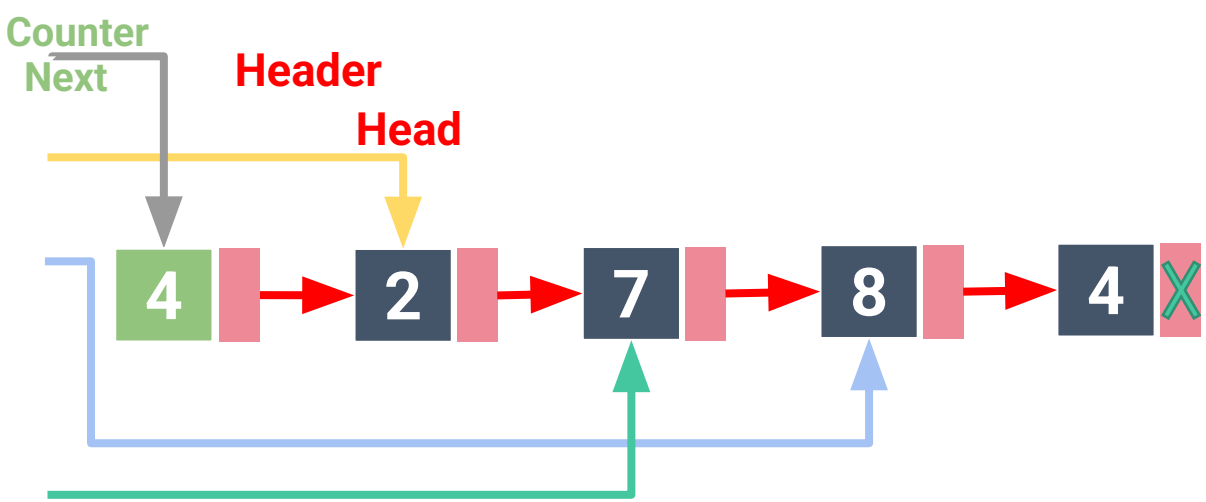
Traversing Singly Linked Lists

Address/ Byte#	Value
6000	4
6001	6002
6002	2
6003	6008
6004	8
6005	6012
6006	
6007	
6008	7
6009	6004
6010	
6011	
6012	4
6013	None

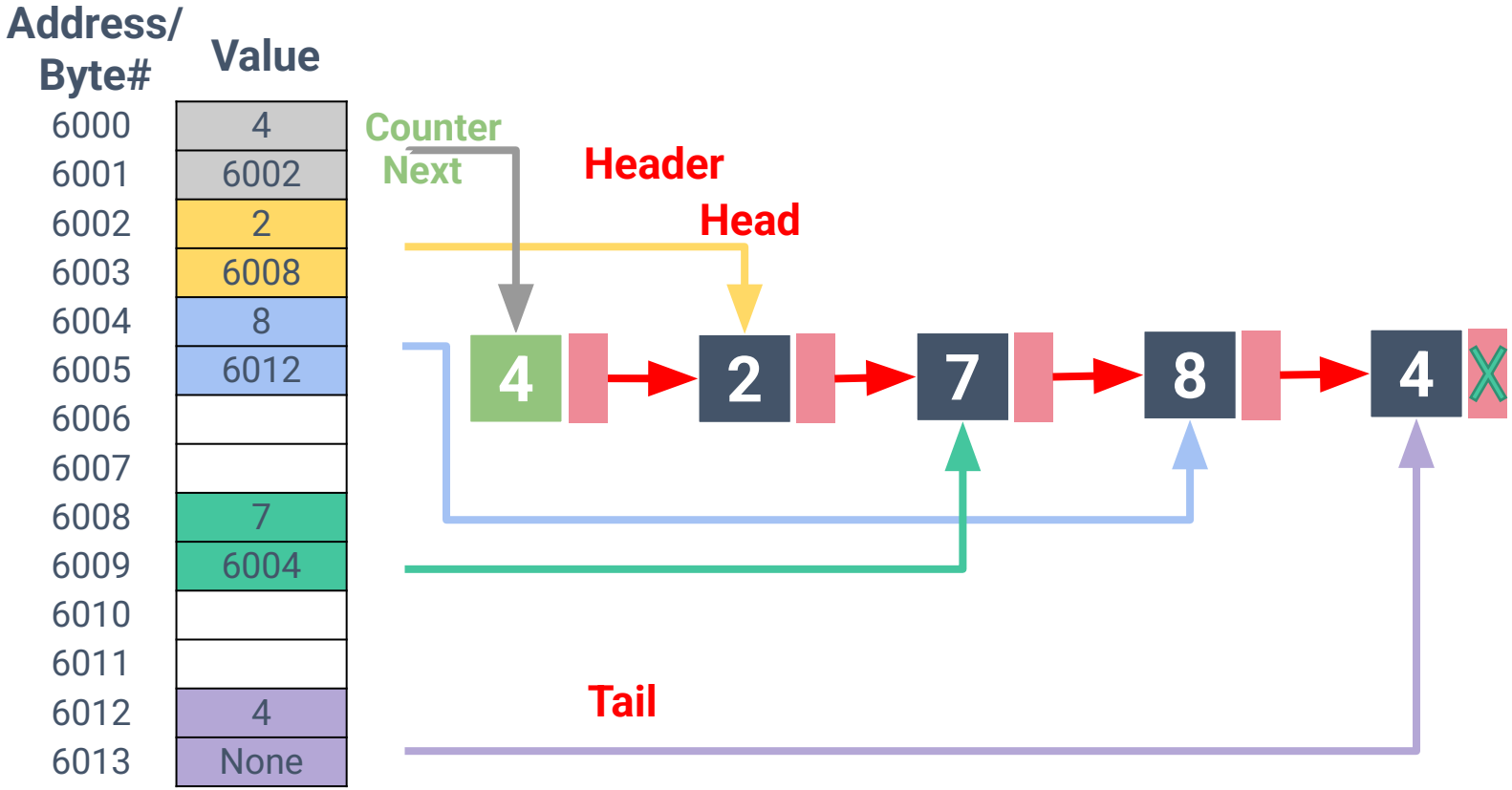


Traversing Singly Linked Lists

Address/ Byte#	Value
6000	4
6001	6002
6002	2
6003	6008
6004	8
6005	6012
6006	
6007	
6008	7
6009	6004
6010	
6011	
6012	4
6013	None

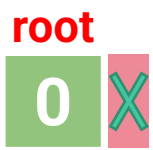


Traversing Singly Linked Lists



Insertion

Add “John” Node into a list



Insertion

at the tail of the list

Add “Tony” Node into a list

Insertion

at the tail of the list

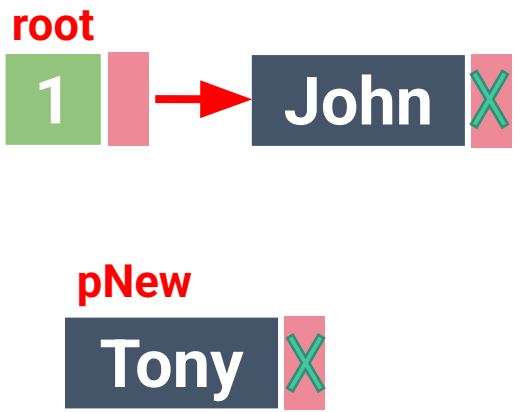
```
Algorithm add_last(L, e):
    new_node = Node(e)                  # Create new node instance
    new_node.next = None                # Set new node's next pointer to None
    if L.tail == None:                  # List was empty
        L.head & L.tail = new_node
    else:
        L.tail.next = new_node          # Make old tail point to new node
        L.tail = new_node               # Update the list's tail to reference the new node
    L.size = L.size + 1                 # Increment the node count
```

$O(1)$

Insertion

at the head of the list

Add “Tony” Node at the front of a list



Insertion

at the head of the list

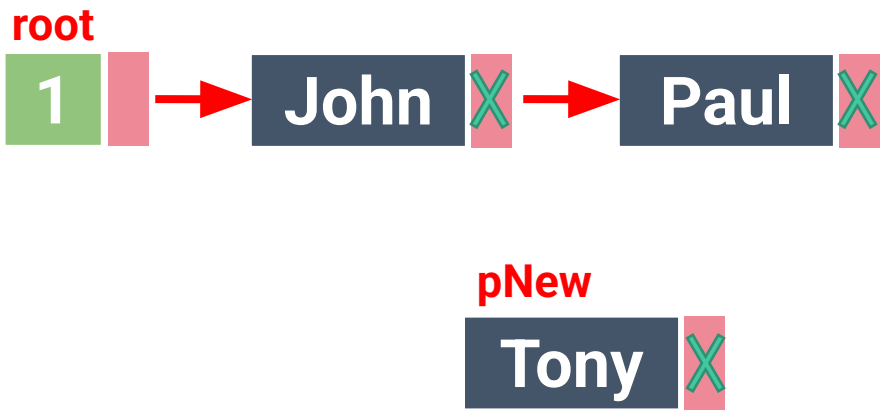
```
Algorithm add_front(L, e):
    new_node = Node(e)              # Create new node instance
    new_node.next = L.head          # Set new node's next pointer to the old head
    L.head = new_node               # Update the list's head to reference the new node
    L.size = L.size + 1             # Increment the node count
    if L.tail == None:              # List was empty
        L.tail = L.head
```

$O(1)$

Insertion

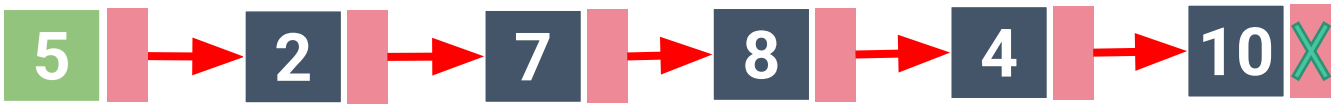
between nodes

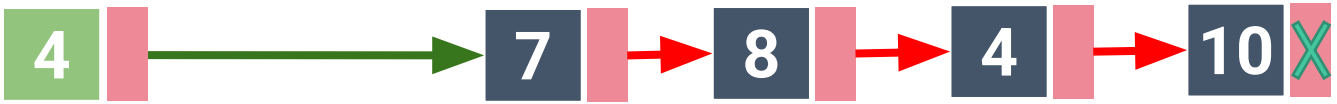
Add “Tony” Node between John and Paul



Deletion

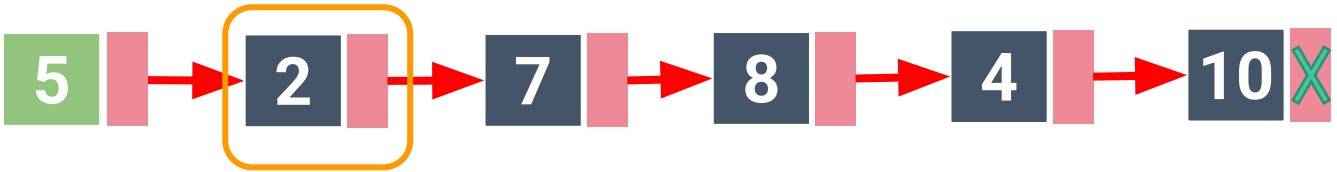
Delete the head node





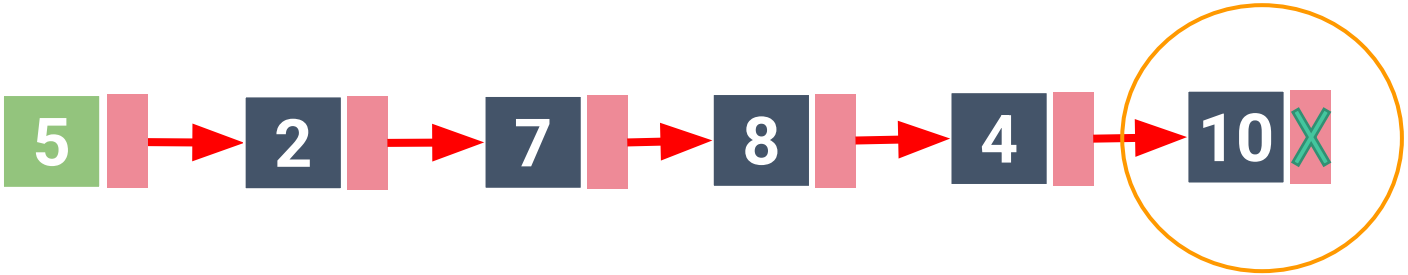
Deletion

Delete the head node



Deletion

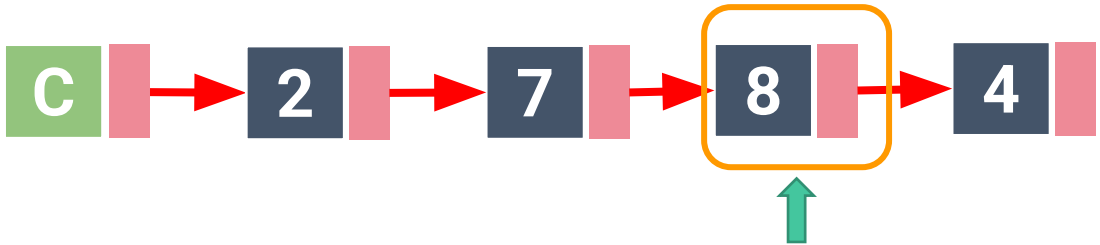
Delete the tail node



Deletion

Delete between nodes

Step 1



Step 2

Step 3